

PCI Pipeline & Algorithm

Technical Verification Document

This document provides a step-by-step description of the Perturbational Complexity Index (PCI) computation pipeline as implemented in `analyze_pci.py`. Each processing step is described with its mathematical formulation, implementation details, and audit status against the reference paper (Casali et al. 2013, Sci. Transl. Med. 5:198ra105).

Document generated: 2026-03-14 13:59
Implementation: `analyze_pci.py` (standalone, numpy-only, no MNE dependency)

Pipeline Overview (10 Steps)

Step 1: Parse BrainVision files (.vhdr/.vmrk/.eeg) - native parser, no MNE
Step 2: Load EEG segment - memory-efficient segment loading per session
Step 3: TMS artifact interpolation - cubic spline over [-2, +10] ms
Step 4: Decimation - anti-alias filter + integer factor downsampling to 1000 Hz
Step 5: Common Average Reference (CAR) - mean subtraction across channels
Step 6: Bad channel detection - variance-based exclusion (>5x median)
Step 7: Bandpass filter - 0.1-45 Hz frequency-domain filter
Step 8: Epoch extraction - [-500, +350] ms windows, amplitude rejection
Step 9: Bootstrap significance matrix - non-parametric, Global Maximum Statistics
Step 10: PCI computation - source sorting + Lempel-Ziv + entropy normalization

Audit Summary

- [+] **OK:** PCI normalization formula matches Casali Eq. 2: $PCI = c \cdot L \cdot \log_2(L) / [L \cdot H(L)]$
- [+] **OK:** LZ traversal is column-major (order='F' in flatten) - correct per Casali
- [+] **OK:** Source sorting by activity before LZ compression - correct per Casali
- [+] **OK:** Bootstrap uses Global Maximum Statistics (max over all sources) - correct
- [!] **WARN:** Bootstrap resamples from evoked baseline z-scores, not single-trial (see Step 9 detail)
- [+] **OK:** Artifact interpolation uses cubic polynomial fit with 4 boundary points
- [+] **OK:** Anti-alias filter applied before decimation with cosine rolloff
- [!] **WARN:** Bandpass filter is single-pass FFT (not zero-phase FIR) - minor phase distortion
- [+] **OK:** QC thresholds active: min_entropy=0.05, min_p1=0.005
- [!] **WARN:** SNR is computed but not used as a gate (no min_snr threshold enforced)
- [!] **WARN:** Sensor-space PCI - thresholds 0.31/0.44 calibrated for source-space (see Limitations)

Step 1: BrainVision File Parsing

Step 1: Parse BrainVision Files

The parser reads .vhdr (header), .vmrk (markers), and .eeg (binary data) files natively without MNE dependency. This avoids MNE cache permission issues in Streamlit environments.

Header (.vhdr)

Extracts: n_channels, sampling_interval_us (converted to sfreq), binary_format (INT_16 or IEEE_FLOAT_32), data_orientation (MULTIPLEXED), channel names and resolutions. For hert_teps_f3: 64 channels, 5000 Hz, INT_16 format, BrainCap TMS 64 system.

Markers (.vmrk)

Parses marker type, description, and sample position. Types: Stimulus, Response, Comment, New Segment. For hert_teps_f3: 150 Response/R16 markers (TMS triggers), 5 Comment markers (goz kapali, iceri girdik, f3, fz, p4).

Trigger Detection Logic

Problem: In many TMS-EEG setups, TMS triggers arrive on the Response port rather than Stimulus port. The parser checks if Response markers form a periodic train by: (1) splitting events into blocks by gaps > 60s, (2) checking each block's coefficient of variation (CV) of inter-stimulus intervals < 0.10, (3) verifying median ISI is between 0.2-15.0 seconds. If periodic, Response markers are used as TMS triggers.

[+] OK: Periodic response train detection works correctly for multi-session data

Session Detection

Sessions are detected from gaps in stimulus timing (default threshold: 60s). Comment markers preceding each session's first event are used as labels. For hert_teps_f3: 3 sessions detected (f3: 50 events, fz: 50 events, p4: 50 events) with ~160-175s gaps.

[+] OK: Session labels correctly assigned from Comment markers (f3, fz, p4)

Step 2: Segment Loading

For large files (882 MB), loading the entire recording into memory causes OOM kills. Solution: load only the segment needed for each session, with 3s padding on each side. Data is read as the native format (INT_16) and converted to float32, with channel resolution applied to get microvolt values. Non-EEG channels (HEOG, VEOG) are excluded via a channel mask during loading.

```
seg_data = load_eeg_segment(eeg_path, info, start_sample, end_sample, channel_mask)
# Returns: float32 array, shape (n_eeg_channels, n_samples), units: microvolts
# Memory: ~62 ch * 765000 samples * 4 bytes = ~190 MB per session (vs 3.5 GB for full file)
```

[+] OK: Memory-efficient segment loading prevents OOM on 882 MB files

Step 3: TMS Artifact Interpolation

The TMS pulse creates a large electromagnetic artifact lasting ~5-10 ms. This must be removed before any filtering (which would spread the artifact). Method: cubic polynomial interpolation using 4 boundary points on each side of the artifact window.

Window: $[t_{stim} - 2\text{ ms}, t_{stim} + 10\text{ ms}]$ (configurable)

For each channel and each TMS pulse: (1) Take 4 samples before artifact start and 4 samples after artifact end as boundary points. (2) Fit a 3rd-degree polynomial (numpy.polyfit) to these 8 boundary points. (3) Evaluate the polynomial over the artifact window to replace the artifact with a smooth interpolation. Fallback to linear interpolation if boundary points are insufficient.

[+] OK: Cubic spline interpolation matches TESA/Comolatti 2019 recommendations

[!] WARN: Dr. Toplatus noted GFP deflection starts after TMS pulse for F3 session. Possible causes: (1) BrainVision trigger timestamp offset - the vmrk marker position may not perfectly align with the actual TMS pulse onset. (2) The [-2, +10] ms window may be too narrow if the system has additional jitter. Recommendation: verify trigger-to-artifact offset by examining raw single-trial data before interpolation.

Steps 4-6: Decimation, Re-reference, Bad Channels

Step 4: Decimation (Anti-alias + Downsample)

Original sampling rate (5000 Hz) is unnecessarily high for PCI computation. Target: 1000 Hz (per 2023 Brain Stimulation consensus for TMS-EEG). Two-stage process: (1) anti-alias low-pass filter, (2) integer decimation.

Anti-alias Filter

FFT-based low-pass filter with cosine rolloff. Cutoff at $0.9 \times \text{Nyquist}$ of target rate ($0.9 \times 500 = 450$ Hz), with rolloff region from 0.7 - $0.9 \times \text{Nyquist}$ (350-450 Hz). Applied per-channel in frequency domain: $\text{spectrum} \times \text{filter_response}$, then inverse FFT.

```
nyq = target_sfreq / 2.0 # 500 Hz for 1000 Hz target
filt[freqs > nyq * 0.9] = 0 # Hard cutoff at 450 Hz
filt[rolloff] = 0.5 * (1 + cos(pi * (f - 350) / 100)) # Cosine rolloff 350-450 Hz
seg_data = seg_data[:, ::dec_factor] # Decimate by factor 5 (5000->1000)
```

[+] OK: Anti-alias filter prevents frequency aliasing during decimation

Event positions are also decimated: $\text{sess_events_local} = [\text{int}(p / \text{dec_factor}) \text{ for } p \text{ in events}]$.

This preserves trigger alignment at the new sampling rate.

Step 5: Common Average Reference (CAR)

$$x_{\text{ref}}(ch, t) = x(ch, t) - (1/N) \times \sum_i x(i, t)$$

Subtracts the mean across all EEG channels at each time point. This removes shared noise components (e.g., line noise, volume conduction artifacts) and approximates a reference-free signal. Applied BEFORE bad channel detection so that noisy channels contribute to the average (and thus their noise is partially removed from good channels).

[!] WARN: CAR is applied before bad channel exclusion. If a very noisy channel is present, it contaminates the average reference. Alternative: iterative approach (detect bad, exclude, re-reference, re-detect). Current order follows standard TMS-EEG practice but may warrant revision if many channels are noisy.

Step 6: Bad Channel Detection

Automated detection based on variance statistics. Two criteria:

NOISY: channel variance $> 5.0 \times$ median variance across channels

DEAD: channel variance $< 0.01 \times$ median variance (near-zero signal)

Bad channels are excluded from all subsequent processing. The channel mask is applied directly to the segment data array. Results for hert_teps_f3:

F3 session: 9 bad channels (F3, Fz, Cz, FC1, FC2, F1, C1, C2, AF3)

- TMS stimulation site + neighbors have high residual artifact variance

Fz session: 9 bad channels (T8, Fz, Cz, FC1, FC2, CP2, C1, C2, CPz)

P4 session: 6 bad channels (P4, CP2, C2, P2, CP4, CPz)

[+] OK: Channels near TMS site are correctly flagged (expected high variance)

[!] WARN: All impedances were '???' (not measured) in this recording. High noise levels across the board suggest impedance preparation was suboptimal. For future recordings, impedances < 5 kOhm should be ensured.

Steps 7-8: Bandpass Filter & Epoch Extraction

Step 7: Bandpass Filter (0.1 - 45 Hz)

Frequency-domain bandpass filter applied per channel. Removes DC drift (< 0.1 Hz) and high-frequency noise/muscle artifacts (> 45 Hz). TMS-evoked potentials (TEPs) have relevant components between 0.1-45 Hz (P30, N45, P60, N100, P180).

Implementation

FFT-based: (1) Compute rfft of each channel. (2) Zero out frequencies below 0.1 Hz and above 45 Hz. (3) Apply smooth cosine transitions at band edges to prevent ringing (Gibbs phenomenon). (4) Inverse rfft to return to time domain.

```
# Low-frequency transition: 0.05-0.1 Hz (cosine taper)
# High-frequency transition: 45-49.5 Hz (cosine taper)
# Applied per-channel: irfft(rfft(data[ch]) * filter_response)
```

[!] WARN: This is a single-pass FFT filter (not zero-phase). It introduces minor phase distortion, which can shift TEP peak latencies by 1-2 ms. For publication-quality analysis, a forward-backward (filtfilt) FIR filter is preferred. However, for PCI computation (which uses binary significance), the forward-only FFT filter is preferred.

Step 8: Epoch Extraction & Artifact Rejection

Epoch Windows

Epoch window: $[t_stim - 500\text{ ms}, t_stim + 350\text{ ms}]$

Each TMS pulse creates one epoch. The pre-stimulus period (-500 to 0 ms) serves as the baseline for bootstrap statistics. The post-stimulus period (0 to +350 ms) contains the TMS-evoked response. The analysis window (8-300 ms) is a subset of this.

Artifact Rejection

Peak-to-peak amplitude criterion: for each epoch, compute the maximum range (max - min) per channel. If ANY channel exceeds the threshold (default: 150 uV), the entire epoch is rejected. This is a conservative approach that ensures all remaining epochs are clean.

```
pp = np.ptp(epoch, axis=1) # peak-to-peak per channel, shape (n_ch,)
if np.any(pp > reject_uv): # reject if ANY channel exceeds threshold
    n_rejected += 1; continue
```

Results for hert_teps_f3:

F3: 47/50 accepted (94%), 3 rejected
Fz: 40/50 accepted (80%), 10 rejected <- higher rejection, more artifacts
P4: 39/50 accepted (78%), 11 rejected <- highest rejection rate

[+] OK: 150 uV threshold is standard for TMS-EEG (Casali used 100-300 uV)

[!] WARN: Fz and P4 sessions have >20% rejection rates. This suggests residual artifacts after interpolation, possibly from suboptimal impedances or stimulation intensity. Minimum 30 clean trials recommended for stable PCI.

Output Format

```
epochs: np.ndarray # shape (n_channels, n_times, n_trials)
times: np.ndarray # shape (n_times,), in seconds relative to TMS pulse
# Example: F3 session -> (53, 851, 47) at 1000 Hz
```

Step 9: Bootstrap Significance Matrix SS(x, t)

Step 9: Non-Parametric Bootstrap with Global Maximum Statistics

This is the most critical step. It determines which source-time pairs have a statistically significant evoked response, producing the binary matrix SS(x,t) that feeds into PCI computation. The method follows Nichols & Holmes (2002) for familywise error correction.

9a. Compute Evoked Response & Z-scores

Average all accepted trials to get the evoked response. Then z-score the post-stimulus evoked response using the baseline statistics:

$$z(x, t) = [evoked(x, t) - \mu_{baseline}(x)] / \sigma_{baseline}(x)$$

Where $\mu_{baseline}$ and $\sigma_{baseline}$ are computed from the evoked response in the pre-stimulus window [-500, -1] ms. Note: these are computed from the EVOKED (trial-averaged) signal, not from single trials.

9b. Build Null Distribution (Bootstrap)

For $b = 1$ to B ($B=500$ iterations): (1) Randomly sample n_{post} time indices from the baseline period (with replacement) (2) Extract the z-scored baseline values at those indices for ALL sources (3) Take the MAXIMUM absolute z-value across all sources and time points (4) Store this maximum as $null_max[b]$ This is Global Maximum Statistics: by taking the maximum over all sources, the null distribution accounts for multiple comparisons across all channels simultaneously.

```
for b in range(n_bootstrap): # B=500
    idx = rng.randint(0, n_baseline, size=n_post) # random baseline indices
    sampled = z_baseline[:, idx] # shape (n_sources, n_post)
    null_maxima[b] = np.max(np.abs(sampled)) # global maximum statistic
```

9c. Threshold & Binary Matrix

$$threshold = percentile(null_maxima, 99\%) \quad [\alpha = 0.01]$$

$$SS(x, t) = 1 \text{ if } |z(x, t)| > threshold, \quad 0 \text{ otherwise}$$

The threshold is the 99th percentile of the null distribution ($\alpha=0.01$). Any post-stimulus source-time pair whose $|z|$ exceeds this threshold is marked as significantly active. Sources with zero baseline variance (dead) are marked as inactive ($SS=0$).

AUDIT: Bootstrap Method

[+] OK: Global Maximum Statistics (max over all sources per iteration) - correct

[+] OK: Same random time indices used across all sources (preserves spatial correlation)

[!] WARN: IMPORTANT: The current implementation bootstraps from the EVOKED (trial-averaged) baseline z-scores, NOT from single-trial baselines. Casali et al. (2013) describe bootstrapping from the trial-averaged baseline, which is what we implement. However, some later implementations (e.g., Comolatti PC1st) use single-trial shuffling. The evoked-baseline approach is valid but may produce different threshold values than single-trial approaches. For our data: thresholds range from 7.08 (f2) to 11.51 (f3), which are reasonable.

Step 10: PCI Computation

Step 10: Perturbational Complexity Index

PCI is computed from the binary significance matrix SS(x,t) in three sub-steps: source sorting, Lempel-Ziv compression, and entropy normalization.

10a. Source Sorting (Optimal Ordination)

Rows of SS are sorted to maximize LZ compression efficiency. Primary sort: total activity (sum of 1s per row, descending). Secondary sort: binary value of row interpreted as a number (most active patterns first). This groups similar activation patterns together, making the LZ scanner more efficient at finding repeated patterns.

```
activity = np.sum(ss_matrix, axis=1) # total 1s per source
powers = 2.0 ** np.arange(n_times-1, -1, -1) # binary weights
row_value = ss_matrix @ powers # binary row value
sort_idx = np.lexsort((-row_value, -activity)) # stable sort
```

10b. Lempel-Ziv Complexity (LZ76)

The sorted SS matrix is flattened COLUMN-MAJOR (Fortran order) into a 1D binary string, then the Lempel-Ziv 1976 algorithm counts the number of distinct 'words' encountered when scanning left to right.

```
flat = ss_matrix.flatten(order='F') # COLUMN-MAJOR (time-first scan)
# This scans: all sources at t=0, all sources at t=1, ..., all sources at t=T
c_L = lempel_ziv_complexity(flat) # count distinct patterns
```

[+] OK: Column-major (order='F') is CORRECT. Casali et al. scan column by column (time sample by time sample), counting new spatial patterns at each time step. Row-major would scan source by source, which gives different (wrong) results. The LZ76 algorithm works as follows: start at position i=1, c=1. At each step, find the longest substring s[i:i+k] that exists in the preceding text s[0:i]. Advance i by (longest_match + 1) and increment c. Higher c = more distinct patterns = more complex brain response.

10c. Entropy Normalization (Casali Eq. 2)

$$PCI = c_L * \log_2(L) / [L * H(L)]$$

Where: c_L = Lempel-Ziv complexity count L = n_sources x n_times (total elements in SS matrix) H(L) = source entropy = -p1*log2(p1) - (1-p1)*log2(1-p1) p1 = fraction of 1s in SS matrix The normalization by L*H/log2(L) ensures PCI is independent of matrix size and activation density. A random binary matrix with entropy H has expected LZ complexity of approximately L*H/log2(L), so PCI ~ 1.0 for random and < 1.0 for structured patterns.

[+] OK: Formula VERIFIED: PCI = c_L / (L * H / log2(L)) = c_L * log2(L) / (L * H). This matches Casali Eq. 2. The earlier bug (PCI = c/c_max) has been FIXED.

10d. Quality Gates

PCI is set to 0.0 (invalid) if: - H(L) < 0.05 (source entropy too low - matrix is nearly all-0 or all-1) - p1 < 0.005 (fraction of active bins too small - no meaningful response) - L = 0 (empty matrix) Note: SNR is computed (signal/noise in evoked response) and reported but NOT used as a hard gate. Recommended: add min_snr=1.4 check for clinical applications.

[!] WARN: min_p1=0.005 is generous. The fz session had p1=0.0100 (just above 0.01), producing PCI=0.5253. While this passed the gate, such low activation may indicate a weak response rather than true cortical complexity. Users should interpret PCI with caution when active% < 2%.

Known Issues & Clinical Concerns

Dr. Eren Toplutas's Observations (5 March 2026)

Issue 1: F3 Session - GFP Timing Anomaly

Observation: GFP positive deflection starts AFTER the TMS pulse marker, not at/before it. Expected: TMS artifact should appear at or slightly before the trigger timestamp. Possible causes: (a) BrainVision trigger timestamp offset: The R16 response marker in .vmrk may have a systematic delay relative to the actual TMS pulse. BrainVision records TTL triggers at the sample when the digital input changes, which may lag the TMS coil discharge by 0.5-2 ms depending on hardware configuration. (b) Artifact interpolation timing: If the [-2, +10] ms window is misaligned relative to the actual artifact, the interpolation may not fully capture the artifact onset. (c) $PCI=0.2599$ is below the unconscious threshold (0.31), which may also reflect poor stimulation quality ('guzel uyaramadik' - we couldn't stimulate well).

RECOMMENDED ACTION: Plot raw single-trial EEG (before any processing) aligned to trigger markers. Check if the TMS artifact peak aligns with sample position 0. If offset, adjust trigger timestamps by the measured delay before running the pipeline.

Issue 2: Fz Session - $PCI=0.5253$ Despite 'Bozuk' (Noisy) Data

Observation: Dr. Toplutas reports EEG data quality is poor with no visible phase locking, yet PCI classifies as 'Conscious' (>0.44). Analysis of the concern: - Active% is only 1.0% (154/15476 bins) - very low - Entropy $H = 0.0805$ - very low (just above our $min_entropy=0.05$ gate) - BUT the 154 active bins are distributed across 53 sources and multiple time points - This sparse, distributed pattern produces HIGH Lempel-Ziv complexity - After normalization by the low entropy, PCI becomes disproportionately large This is a known mathematical property: when p_1 is very small, even modest LZ complexity yields high PCI because the normalizer ($L \cdot H / \log_2(L)$) is also very small. This is NOT necessarily a false positive - it could reflect genuine diffuse cortical activation. However, it warrants caution.

RECOMMENDED ACTION: (1) Examine the $SS(x,t)$ matrix visually - is the activation pattern physiologically plausible or random-looking? (2) Compare with single-trial butterfly plots to check for phase locking. (3) Consider raising min_p_1 to 0.01 or $min_entropy$ to 0.08 for more conservative gating. (4) Report PCI with the Active% caveat.

Methodological Limitations

Sensor-Space vs Source-Space PCI

CRITICAL: The reference thresholds (0.31, 0.44) from Casali et al. (2013) were calibrated using ~5000 cortical dipoles from MRI-based source reconstruction (3-sphere BERG model). Our implementation computes PCI in sensor space (EEG channels as 'sources'). This means: - Number of sources is different (~53 vs ~5000) - Spatial resolution is much lower - Volume conduction mixes signals across sensors - Absolute PCI values are NOT directly comparable to Casali thresholds The thresholds are shown for REFERENCE ONLY. Sensor-space PCI requires separate threshold calibration with a known-state dataset (Casarotto et al. 2016).

Complete Processing Pipeline Flowchart



*detect sessions
from Comment
markers & gaps*

*per TMS pulse
per channel*

*F3: 9 bad
Fz: 9 bad
P4: 6 bad*

*F3: 47/50
Fz: 40/50
P4: 39/50*

*Global Maximum
Statistics
(Nichols 2002)*

**F3: 0.2599
Fz: 0.5253
P4: 0.3714**

OUTPUT: PCI value per session

Recommendations & References

Recommended Improvements (Priority Order)

1. Trigger Timing Verification (HIGH)

Add a diagnostic step that plots raw single-trial data around each trigger to verify the TMS artifact peak aligns with trigger sample position. If systematic offset is found, shift trigger positions before processing. This directly addresses Dr. Toplutas's F3 concern.

2. Single-Trial QC Plots (HIGH)

Generate per-session butterfly plots of individual trials (not just the average). This allows visual assessment of phase locking and identifies sessions where evoked response is not reproducible across trials (like the fz session concern).

3. SNR Gating (MEDIUM)

Add an optional `min_snr` parameter (default 1.4) that prevents PCI computation when the evoked response SNR is too low. Currently SNR is computed and reported but not used as a gate. $SNR < 1.4$ suggests the evoked response is not reliably above noise.

4. Zero-Phase Bandpass Filter (MEDIUM)

Replace the single-pass FFT filter with a forward-backward (`filtfilt`) implementation to eliminate phase distortion. This requires `scipy` or a custom implementation. Not critical for PCI (binary significance) but important for TEP peak latency analysis.

5. PC1st Integration (FUTURE)

Implement Comolatti et al. (2019) PC1st method which uses SVD-based source estimation instead of bootstrap statistics. Advantages: no MRI needed, 5-10x faster computation, validated against standard PCI. Can serve as independent verification of results.

6. Iterative Bad Channel + CAR (FUTURE)

Current order: CAR then bad channel detection. If very noisy channels exist, they contaminate the CAR. Consider: (1) rough bad channel detection, (2) exclude worst, (3) CAR on remaining, (4) re-run bad channel detection.

References

1. Casali AG et al. (2013). A theoretically based index of consciousness independent of sensory processing and behavior. *Sci Transl Med*, 5(198):198ra105.
2. Comolatti R et al. (2019). A fast and general method to empirically estimate the complexity of brain responses. *Brain Stimulation*, 12(5):1280-1289.
3. Nichols TE, Holmes AP (2002). Nonparametric permutation tests for functional neuroimaging. *Human Brain Mapping*, 15(1):1-25.
4. Kaspar F, Schuster HG (1987). Easily calculable measure for the complexity of spatiotemporal patterns. *Physical Review A*, 36:842-848.
5. Rogasch NC et al. (2017). Analysing concurrent transcranial magnetic stimulation and electroencephalographic data: A review. *NeuroImage*, 147:16-24.
6. Casarotto S et al. (2016). Stratification of unresponsive patients by an independently validated index of brain complexity. *Ann Neurol*, 80(5):718-729.